Experiment 05: Containerizing Applications using Docker

Experiment No.	05
Mapped CO	CO4
Aim	To containerize an application by writing a Dockerfile, building an image and running it as a container.
Tools / Software	Docker Engine; A sample web application such as Python Flask or Node.js; Docker Hub account optional; Code editor / terminal

Theory

A container is an isolated and runnable instance of an application packaged with its required dependencies. Containers share the host operating-system kernel and therefore start faster and consume fewer resources than traditional virtual machines. Docker is a commonly used platform for building, running and managing containers.

A Docker image is an immutable, layered template created from instructions written in a Dockerfile. Common Dockerfile instructions include FROM, WORKDIR, COPY, RUN, EXPOSE and CMD. When an image is executed, it becomes a container. Images can be tagged and stored in a registry such as Docker Hub for sharing and deployment.

Procedure

1. Install Docker Engine and verify the installation using docker --version.
2. Create a project directory containing a sample web application file and a dependency file such as requirements.txt.
3. Write a Dockerfile defining the base image, working directory, dependency installation, port exposure and start command.
4. Build the Docker image using docker build and assign a meaningful tag such as myapp:1.0.
5. Run the image as a container using docker run and map the container port to a host port.
6. List running containers using docker ps and inspect container logs using docker logs.
7. Open the application in a browser or use curl to verify that it is reachable on the mapped port.
8. Optionally tag and push the image to Docker Hub for sharing.

Sample Code / Commands

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return "Hello from Docker Container"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Build, run and publish commands

```
docker --version
docker build -t myapp:1.0 .
docker run -d -p 5000:5000 --name myapp myapp:1.0
docker ps
docker logs myapp
curl http://localhost:5000
docker tag myapp:1.0 <user>/myapp:1.0
docker push <user>/myapp:1.0
```

Observation / Experiment Output

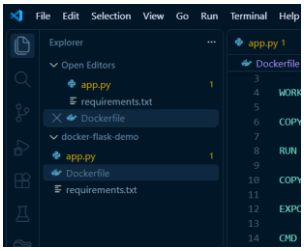
Instructions to students: Paste screenshots or copied outputs in the boxes below after completing Docker installation verification, application creation, image building, container execution and output validation. Keep screenshots clear and readable.

Name	Roll No.	Batch	Date
Krishna Vijay Nikam	43	B3	

1. Docker Installation and Version Output

```
C:\Users\Vijay>docker --version
Docker version 29.8.2, build 7fc2dff
```

2. Application Source and Dependency Files Output



3. Dockerfile Output

```
C:\Users\Vijay\docker-flask-demo>docker images
IMAGE       ID             DISK USAGE   CONTENT SIZE   EXTRA
myapp:1.0   f7e17ed47e9a        211MB         51.3MB

C:\Users\Vijay\docker-flask-demo>
```

i Info → U In Use

4. Docker Image Build Output

```
C:\Users\Vijay\docker-flask-demo>docker ps
CONTAINER ID   IMAGE        COMMAND           CREATED              STATUS              PORTS
          NAMES
e29fdf148869   myapp:1.0    "python app.py"   About a minute ago   Up About a minute   0.0.0.0:5000->5000/tcp, [::]:5000-
>5000/tcp   flask-container
```

5. Container Run, Logs and Application Access Output



Flask Application Running Successfully in Docker!

6. Docker Hub Push Output (Optional)

```
C:\Users\Vijay\docker-flask-demo>docker stop flask-container
flask-container

C:\Users\Vijay\docker-flask-demo>docker rm flask-container
flask-container

C:\Users\Vijay\docker-flask-demo>docker ps -a
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

GitHub Repository / Lab Folder Link: https://github.com/nikamkrishna055-coder/EXP-5-

Docker Hub Repository Link (Optional):

Final Observation Statement:

Write 2-3 lines summarizing the result of the experiment.

Expected Output

The Docker image should build successfully, the container should run in the background, the application should be reachable on the mapped host port, and docker ps should list the running container. If Docker Hub is used, the tagged image should be visible in the repository.

Conclusion

The application was containerized by writing a Dockerfile, building a Docker image and running it as an isolated portable container, demonstrating image creation, container execution, port mapping and basic container lifecycle management.

Viva / Review Questions

1. Differentiate a container from a virtual machine.

2. What is the difference between a Docker image and a Docker container?
3. Explain the purpose of EXPOSE in a Dockerfile and the -p flag in docker run.
4. Why are Docker image layers cached?

Marks: ____ / 10 Date: _____ Signature of Faculty: _____